

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS ARARANGUÁ - ENGENHARIA DE COMPUTAÇÃO
DEC7129 - Banco de dados I

Manuel Etiene da Silva João
Mateus Mendes Sodré
Zeferino José Paulo Fulano

Trabalho Final: Plataforma de acompanhamento de conquistas de jogos

Professor: Alexandre Leopoldo Gonçalves
22/11/2026

Sumário

1. Objetivo Geral do Sistema....3

2. Descrição Detalhada do Sistema....4

2.1 Contexto e escopo....4

2.2 Principais elementos do domínio....4

2.3 Regras de negócio....5

2.4 Requisitos funcionais....6

2.5 Requisitos não funcionais....7

3. Modelo Conceitual....8

4. Modelo Lógico....9

4.1 Obtenção do Modelo....9

4.2 Regras de Mapeamento....9

4.3 Descrição das tabelas....10

4.4 Restrições de Integridade....11

4.5 Políticas de exclusão....12

4.6 Regras implementadas por gatilhos (triggers)....12

5. Script DDL....14

5.1 Organização dos scripts....14

5.2 Criação das tabelas (01_schema.sql)....14

5.3 Suporte a embeddings (02_pgvector.sql)....21

6. Consultas....22

6.1 Ranking de gamerscore por usuário....22

6.2 Conquistas desbloqueadas por trimestre....23

6.3 Jogos concluídos (100% do pacote base) por ano....25

7. Aplicação....27

7.1 Tecnologias e execução....27

7.2 Funcionalidades....27

7.3 Recomendação com IA generativa....28

1. Objetivo Geral do Sistema

O projeto tem como objetivo o desenvolvimento de um sistema de banco de dados e de uma aplicação para o acompanhamento de jogos eletrônicos e de conquistas no ecossistema Xbox, permitindo que os usuários organizem sua biblioteca pessoal de jogos e acompanhem seu progresso individual. O sistema armazenará informações sobre jogos, plataformas, gêneros, desenvolvedoras, publicadoras e pacotes de conquistas (jogo base, conteúdos adicionais e atualizações), além de registrar as conquistas desbloqueadas por cada usuário, com data e hora de obtenção, e o tempo dedicado a cada jogo. A partir desses dados, será possível calcular o progresso por jogo, a pontuação acumulada (gamerscore) e gerar estatísticas e classificações que permitam analisar o desempenho e os hábitos de jogo de cada usuário. Adicionalmente, o sistema utilizará Inteligência Artificial Generativa para recomendar novos jogos, tomando como base os dados armazenados e as preferências identificadas no histórico do usuário.

2. Descrição Detalhada do Sistema

2.1 Contexto e escopo

Plataformas de jogos eletrônicos recompensam os jogadores com conquistas, que funcionam como marcos de progresso dentro de cada título. No ecossistema Xbox, cada conquista possui uma pontuação chamada *gamerscore* (G), e um jogo completo (jogo base) soma exatamente 1000 G quando todas as suas conquistas são obtidas. Conteúdos adicionais (DLCs) e atualizações (*updates*) podem acrescentar novas conquistas em "pacotes" separados, que rendem pontuação extra além dos 1000 G do jogo base. Além disso, um mesmo título pode ter listas de conquistas distintas conforme a plataforma em que é jogado (por exemplo, Xbox 360, Xbox One, Xbox Series X|S e PC).

O sistema modelará esse domínio de forma simplificada. **Estão no escopo:** o cadastro de usuários; o catálogo de jogos, plataformas, gêneros, desenvolvedoras e publicadoras; a organização das conquistas em pacotes por jogo e plataforma; a biblioteca pessoal de cada usuário; o registro dos desbloqueios; o cálculo de progresso e pontuação; as consultas estatísticas com representação gráfica; a recomendação de jogos por IA generativa; e uma interface web local (Streamlit) para operar o sistema. **Estão fora do escopo:** a integração automática com serviços externos (como a API do Xbox Live), autenticação por senha, compra ou licenciamento de jogos e DLCs (a posse de conteúdos adicionais pelo usuário não é controlada) e funcionalidades sociais. Os dados utilizados serão fictícios, carregados por script, com possibilidade de inclusão manual pela aplicação.

2.2 Principais elementos do domínio

- **Usuário:** pessoa cadastrada na plataforma, identificada por e-mail e nome de exibição (gamertag) únicos, com data de cadastro registrada automaticamente.
- **Jogo:** título com data de lançamento, descrição, uma desenvolvedora e uma publicadora, um ou mais gêneros e disponibilidade em uma ou mais plataformas.
- **Desenvolvedora e Publicadora:** empresas cadastradas separadamente, o que permite listar os jogos desenvolvidos por uma desenvolvedora e os publicados por uma publicadora.
- **Plataforma e Gênero:** classificações do catálogo (por exemplo, Xbox One, Xbox Series X|S; Ação, RPG, Corrida).
- **Jogo_Plataforma:** representa a existência de um jogo em uma plataforma específica. É a referência para pacotes de conquistas e para a biblioteca, pois as listas de conquistas podem diferir entre plataformas.

- **Pacote de conquistas:** agrupador de conquistas de um jogo em uma plataforma, com tipo BASE, DLC ou UPDATE. Todo jogo por plataforma possui exatamente um pacote base e zero ou mais pacotes adicionais.
- **Conquista:** possui nome, descrição e gamerscore, e pertence a um único pacote.
- **Item_Biblioteca:** associa um usuário a um jogo em uma plataforma, registrando status, horas jogadas e data de inclusão. A biblioteca do usuário é o conjunto dos seus itens.
- **Desbloqueio:** registra que uma conquista foi obtida por um usuário (em determinada entrada da biblioteca), com data e hora.
- **Recomendação:** sugestão de jogos gerada por IA para um usuário, com data de geração e modelo utilizado. Lista um ou mais jogos, cada um com justificativa e posição no ranking.

2.3 Regras de negócio

RN01 - O e-mail e o nome de exibição de cada usuário são únicos.

RN02 - A data de cadastro do usuário é registrada automaticamente.

RN03 - Todo jogo possui título, data de lançamento, exatamente uma desenvolvedora e exatamente uma publicadora.

RN04 - Um jogo possui ao menos um gênero e ao menos uma plataforma de disponibilidade.

RN05 - Uma desenvolvedora ou publicadora pode estar associada a vários jogos.

RN06 - Cada jogo por plataforma possui exatamente um pacote de conquistas BASE e zero ou mais pacotes DLC ou UPDATE.

RN07 - Toda conquista pertence a um único pacote e possui gamerscore inteiro maior que zero. *(Simplificação: no Xbox real há exceções, como conquistas de 0 G.)*

RN08 - A soma do gamerscore das conquistas do pacote BASE deve ser igual a 1000 G. *(Regra adotada no projeto.)*

RN09 - A soma do gamerscore de um pacote DLC ou UPDATE não excede o limite de 250 G *(simplificação adotada no projeto).*

RN10 - Ao adicionar um jogo à biblioteca, o usuário informa a plataforma, que deve ser uma das plataformas em que o jogo está disponível.

RN11 - O mesmo jogo pode constar na biblioteca de um usuário mais de uma vez, desde que em plataformas diferentes.

RN12 - O status do jogo na biblioteca assume um dos valores: QUERO_JOGAR, JOGANDO, PAUSADO, ABANDONADO ou FINALIZADO.

RN13 - As horas jogadas são um valor numérico maior ou igual a zero.

RN14 - O usuário só desbloqueia conquistas de jogos presentes em sua biblioteca e da plataforma correspondente.

RN15 - Cada conquista só pode ser desbloqueada uma vez por entrada da biblioteca, com data e hora obrigatórias, e a data/hora do desbloqueio não pode ser futura.

RN16 - O progresso de um pacote (%) é a razão entre as conquistas desbloqueadas e o total de conquistas do pacote.

RN17 - Um jogo é considerado concluído quando o pacote base atinge 100%. A conclusão de DLCs e updates é um indicador separado.

RN18 - O gamerscore do usuário é a soma das conquistas desbloqueadas em todos os pacotes, podendo ultrapassar 1000 G por jogo.

RN19 - Não é permitido excluir gênero, plataforma, desenvolvedora ou publicadora vinculados a jogos.

RN20 - A exclusão de um jogo remove em cascata seus vínculos com gêneros e plataformas, seus pacotes, conquistas, entradas de biblioteca, desbloqueios e vínculos com recomendações.

RN21 - A exclusão de um usuário remove sua biblioteca, seus desbloqueios e suas recomendações.

RN22 - Cada recomendação pertence a um único usuário, registra a data de geração e o modelo de IA utilizado, e lista ao menos um jogo.

RN23 - Em cada recomendação, cada jogo recomendado possui uma justificativa e uma posição (ordem no ranking), e um mesmo jogo não aparece duas vezes na mesma recomendação.

2.4 Requisitos funcionais

RF01 - Manter (inserir, atualizar, excluir e consultar) usuários, jogos, plataformas, gêneros, desenvolvedoras e publicadoras.

RF02 - Manter os vínculos entre jogos e gêneros e entre jogos e plataformas.

RF03 - Manter pacotes de conquistas e conquistas.

RF04 - Adicionar, atualizar (status, horas jogadas) e remover jogos da biblioteca do usuário.

RF05 - Registrar, alterar e remover o desbloqueio de conquistas.

RF06 - Calcular o progresso por pacote e por jogo, e o gamerscore acumulado do usuário.

RF07 - Listar a biblioteca, os jogos concluídos e as conquistas de um usuário.

RF08 - Listar os jogos desenvolvidos por uma desenvolvedora e os publicados por uma publicadora.

RF09 - Gerar classificações de usuários por gamerscore, conquistas e jogos concluídos.

RF10 - Executar as consultas estatísticas do item 6, exibindo o resultado em forma tabular e gráfica.

RF11 - Recomendar jogos por meio de IA generativa, com base nos gêneros, nas horas jogadas e no histórico do usuário.

RF12 - Criar as tabelas e carregar dados pré-definidos a partir de um banco vazio, e eliminar todas as tabelas.

RF13 - Disponibilizar um menu principal que dê acesso a todas as funcionalidades.

2.5 Requisitos não funcionais

RNF01 - O sistema utiliza o SGBD PostgreSQL.

RNF02 - A aplicação possui interface web (Streamlit), executada localmente e conectada diretamente ao PostgreSQL.

RNF03 - A integridade dos dados é garantida pelo banco (chaves primárias e estrangeiras, UNIQUE, CHECK e, quando necessário, triggers), e não apenas pela aplicação.

RNF04 - A carga inicial é reprodutível a partir de um banco vazio.

RNF05 - A chave de acesso ao serviço de IA é configurada fora do código-fonte (variável de ambiente).

Obs.: *Para fins didáticos, o sistema adota as seguintes simplificações em relação ao ecossistema Xbox real: (i) todas as conquistas possuem gamerscore maior que zero; (ii) o pacote base de cada jogo por plataforma soma exatamente 1000 G; (iii) cada pacote de DLC ou update soma no máximo 250 G; (iv) a posse de conteúdos adicionais pelo usuário não é controlada; (v) os dados do catálogo e dos desbloqueios são fictícios; (vi) variantes regionais não são modeladas, porque Jogo_Plataforma identifica o par jogo × plataforma.*

3. Modelo Conceitual

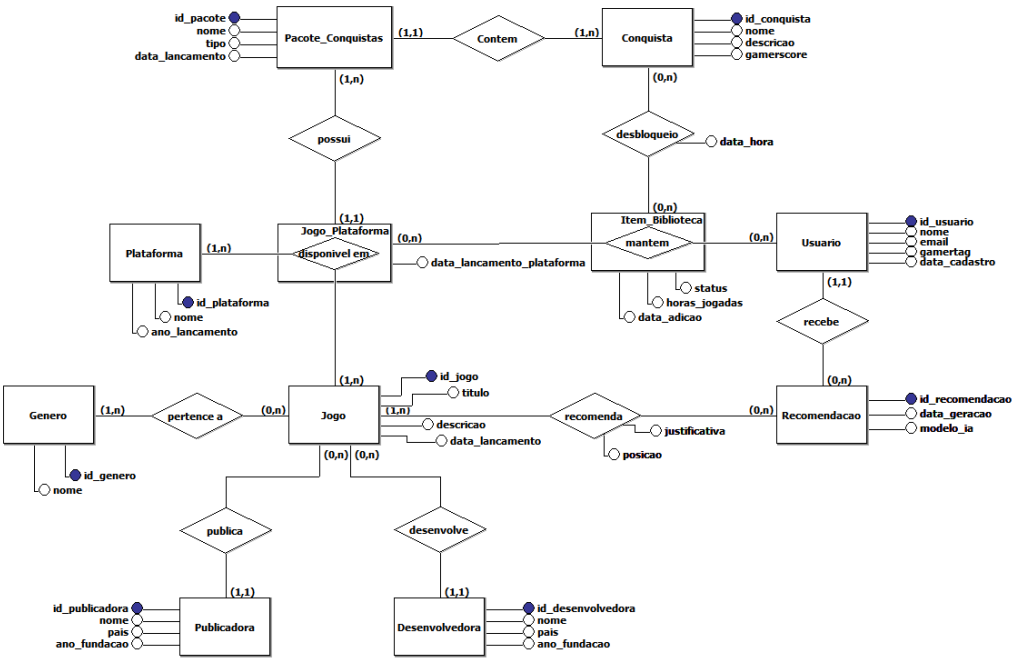


Figura 1: Diagrama ER

4. Modelo Lógico

4.1 Obtenção do Modelo

O modelo lógico foi gerado a partir do modelo conceitual (seção 3) com o auxílio da ferramenta brModelo, seguido de ajustes manuais. Os ajustes foram necessários porque a ferramenta não propaga as chaves das entidades associativas às tabelas que dependem delas, o que deixava Pacote_Conquistas, Item_Biblioteca e Desbloqueio sem as chaves estrangeiras correspondentes. Foram também definidos os tipos de dados de todas as colunas.

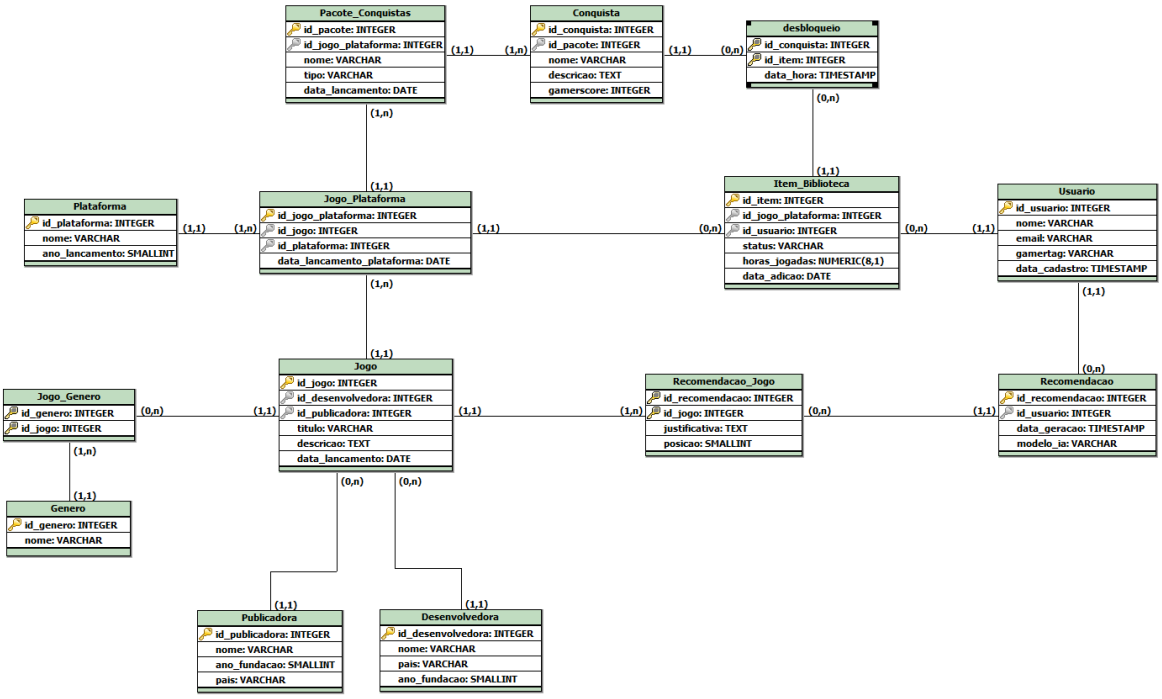


Figura 2: Diagrama Lógico

4.2 Regras de Mapeamento

- Cada entidade do modelo conceitual originou uma tabela, e o atributo identificador tornou-se a chave primária.
- Relacionamentos 1:N foram implementados por chave estrangeira no lado N.
- Relacionamentos N:N originaram tabelas próprias, com chave primária composta pelas chaves das entidades relacionadas e com os atributos do relacionamento: Jogo_Genero (pertence a), Desbloqueio (desbloqueio) e Recomendacao_Jogo (recomenda).
- As entidades associativas Jogo_Plataforma e Item_Biblioteca foram implementadas com chave substituta (id_jogo_plataforma e id_item). Como outras tabelas as referenciam, a chave substituta evita propagar chaves compostas de duas ou mais colunas e simplifica as chaves estrangeiras. A unicidade do par original é mantida por restrição UNIQUE (seção 4.4).

O modelo conceitual, com 11 entidades, resultou em **14 tabelas**: as 11 entidades mais as 3 tabelas oriundas de relacionamentos N:N.

4.3 Descrição das tabelas

Tabela	PK	FK	Demais colunas
Usuario	id_usuario (INTEGER)	-	nome (VARCHAR), email (VARCHAR), gamertag (VARCHAR), data_cadastro (TIMESTAMP)
Desenvolvedora	id_desenvolvedora (INTEGER)	-	nome (VARCHAR), pais (VARCHAR), ano_fundacao (SMALLINT)
Publicadora	id_publicadora (INTEGER)	-	nome (VARCHAR), pais (VARCHAR), ano_fundacao (SMALLINT)
Genero	id_genero (INTEGER)	-	nome (VARCHAR)
Plataforma	id_plataforma (INTEGER)	-	nome (VARCHAR), ano_lancamento (SMALLINT)
Jogo	id_jogo (INTEGER)	id_desenvolvedora, id_publicadora	titulo (VARCHAR), descricao (TEXT), data_lancamento (DATE)
Jogo_Genero	(id_jogo, id_genero)	id_jogo, id_genero	-
Jogo_Plataforma	id_jogo_plataforma (INTEGER)	id_jogo, id_plataforma	data_lancamento_plataforma (DATE)
Pacote_Conquistas	id_pacote (INTEGER)	id_jogo_plataforma	nome (VARCHAR), tipo (VARCHAR), data_lancamento (DATE)
Conquista	id_conquista (INTEGER)	id_pacote	nome (VARCHAR), descricao (TEXT), gamerscore (INTEGER)
Item_Biblioteca	id_item (INTEGER)	id_usuario, id_jogo_plataforma	status (VARCHAR), horas_jogadas (NUMERIC(8,1)), data_adicao (DATE)
Desbloqueio	(id_item, id_conquista)	id_item, id_conquista	data_hora (TIMESTAMP)
Recomendacao	id_recomendacao (INTEGER)	id_usuario	data_geracao (TIMESTAMP), modelo_ia (VARCHAR)
Recomendacao_Jogo	(id_recomendacao, id_jogo)	id_recomendacao, id_jogo	justificativa (TEXT), posicao (SMALLINT)

Observação: *o progresso (%), o gamerscore acumulado do usuário e a condição de "jogo concluído" são atributos derivados e, por isso, não são armazenados: são calculados por consulta, o que evita inconsistências.*

4.4 Restrições de Integridade

Além das chaves, o banco impõe as restrições abaixo. A coluna "Regra" indica a regra de negócio da seção 2.3 atendida.

Todas as colunas dos atributos obrigatórios do modelo conceitual são declaradas NOT NULL; são opcionais apenas pais e ano_fundacao (desenvolvedora e publicadora), ano_lancamento (plataforma), descricao (jogo) e data_lancamento_plataforma. A tabela a seguir destaca as demais restrições.

Tabela	Restrição	Regra
Usuario	UNIQUE (email); UNIQUE (gamertag)	RN01
Usuario	data_cadastro com valor padrão now()	RN02
Jogo	id_desenvolvedora e id_publicadora NOT NULL	RN03
Genero	UNIQUE (nome)	-
Plataforma	UNIQUE (nome)	-
Jogo_Plataforma	UNIQUE (id_jogo, id_plataforma)	-
Pacote_Conquistas	CHECK (tipo IN ('BASE','DLC','UPDATE'))	RN06
Pacote_Conquistas	Índice único parcial sobre id_jogo_plataforma com WHERE tipo = 'BASE' (no máximo um pacote base por jogo/plataforma)	RN06
Conquista	CHECK (gamerscore > 0)	RN07
Item_Biblioteca	FK id_jogo_plataforma → Jogo_Plataforma (estrutural)	RN10
Item_Biblioteca	UNIQUE (id_usuario, id_jogo_plataforma)	RN11
Item_Biblioteca	CHECK (status IN ('QUERO_JOGAR','JOGANDO','PAUSADO','ABANDONADO','FINALIZADO'))	RN12
Item_Biblioteca	CHECK (horas_jogadas >= 0)	RN13
Desbloqueio	Chave primária composta (id_item, id_conquista)	RN15
Desbloqueio	data_hora NOT NULL	RN15
Recomendacao_Jogo	Chave primária composta (id_recomendacao, id_jogo)	RN23
Recomendacao_Jogo	UNIQUE (id_recomendacao, posicao)	RN23
Recomendacao_Jogo	posicao NOT NULL e CHECK (posicao > 0)	RN23
Recomendacao_Jogo	justificativa NOT NULL	RN23

4.5 Políticas de exclusão

Chave estrangeira	Política	Regra
Jogo → Desenvolvedora	RESTRICT	RN19
Jogo → Publicadora	RESTRICT	RN19
Jogo_Genero → Genero	RESTRICT	RN19
Jogo_Plataforma → Plataforma	RESTRICT	RN19
Jogo_Genero → Jogo	CASCADE	RN20
Jogo_Plataforma → Jogo	CASCADE	RN20
Pacote_Conquistas → Jogo_Plataforma	CASCADE	RN20
Conquista → Pacote_Conquistas	CASCADE	RN20
Item_Biblioteca → Jogo_Plataforma	CASCADE	RN20
Desbloqueio → Conquista	CASCADE	RN20
Recomendacao_Jogo → Jogo	CASCADE	RN20
Item_Biblioteca → Usuario	CASCADE	RN21
Desbloqueio → Item_Biblioteca	CASCADE	RN20, RN21
Recomendacao → Usuario	CASCADE	RN21
Recomendacao_Jogo → Recomendacao	CASCADE	RN21

4.6 Regras implementadas por gatilhos (triggers)

Algumas regras de negócio envolvem mais de uma tabela ou dependem do estado dos dados, e não podem ser expressas por restrições declarativas simples. Elas são implementadas por gatilhos no banco ou, quando indicado, pela aplicação.

Código	Regra	Verificação	Implementação
T1	RN14	Em Desbloqueio, a conquista deve pertencer, por meio de seu pacote, ao mesmo Jogo_Plataforma do item da biblioteca.	Gatilho BEFORE INSERT OR UPDATE
T2	RN15	A data_hora do desbloqueio não pode ser futura.	Gatilho BEFORE INSERT OR UPDATE

Código	Regra	Verificação	Implementação
T3	RN08, RN09	A soma do gamerscore de um pacote BASE não excede 1000 G, e a de um pacote DLC/UPDATE não excede 250 G. A igualdade a 1000 G no pacote base é verificada por consulta de auditoria.	Gatilho em Conquista + consulta de auditoria
T4	RN04, RN06, RN22	Todo jogo possui ao menos um gênero e uma plataforma, todo jogo por plataforma possui um pacote base, e toda recomendação lista ao menos um jogo.	Aplicação

A igualdade exata (1000 G) e as condições de “ao menos um” não podem ser impostas a cada inserção isolada, pois o primeiro registro inserido nunca satisfaria a regra. Por isso, o banco impõe o limite máximo, e a completude é garantida pela aplicação e verificada por consulta de auditoria. O gatilho T2 foi preferido a um CHECK com now(), pois uma restrição cujo resultado muda com o tempo é inadequada, por exemplo, na restauração de cópias do banco.

5. Script DDL

O banco é criado a partir de scripts SQL versionados no repositório do projeto (pasta `sql/`), executados em ordem por um único comando da aplicação (seção 7). A tabela a seguir resume cada arquivo.

5.1 Organização dos scripts

Arquivo	Conteúdo	Origem
01_schema.sql	14 tabelas, chaves, restrições, índices, gatilhos T1 a T3 e a visão de auditoria vw_auditoria_integridade	Escrito à mão (seção 5.2)
02_pgvector.sql	Extensão pgvector e as colunas embedding (vector 768) e url_capa em jogo	Escrito à mão (seção 5.3)
03_carga.sql	Dados fictícios: 5 usuários, 15 jogos, 18 jogo/plataforma, 26 pacotes, 213 conquistas, 28 itens de biblioteca, 225 desbloqueios (613 linhas no total)	Escrito à mão
04_embeddings.sql	Um UPDATE por jogo com o vetor de embedding	Gerado por gerar_embeddings.py
05_capas.sql	Endereços das capas dos jogos (opcional)	Gerado por capas.py
99_drop.sql	Elimina a visão, as tabelas e as funções	Escrito à mão

A ordem de execução para montar o banco a partir do zero é 01, 02, 03, 04 e 05 (os dois últimos são opcionais), e 99 para eliminar tudo. As tabelas são criadas dos pais para os filhos e eliminadas na ordem inversa, respeitando as dependências de chave estrangeira.

5.2 Criação das tabelas (01_schema.sql)

O script implementa integralmente o modelo lógico da seção 4: chaves primárias e estrangeiras, as restrições da seção 4.4, as políticas de exclusão da seção 4.5 e os gatilhos da seção 4.6, com índices nas chaves estrangeiras.

```
-- ======================================================================
-- Plataforma de Acompanhamento de Conquistas de Jogos (Xbox)
-- DEC7129 - Banco de Dados I - UFSC Araranguá
-- Script DDL (PostgreSQL) - tabelas, restrições, índices, gatilhos e
-- visão de auditoria. Gerado a partir do modelo lógico do relatório.
-- ======================================================================

-- ----------------------------------------------------------------------
-- 1. TABELAS
-- ----------------------------------------------------------------------

CREATE TABLE usuario (
    id_usuario     INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nome           VARCHAR(100) NOT NULL,
```

```
    email          VARCHAR(150) NOT NULL,
    gamertag       VARCHAR(30)  NOT NULL,
    data_cadastro  TIMESTAMP    NOT NULL DEFAULT now(),          -- RN02
    CONSTRAINT uq_usuario_email    UNIQUE (email),               -- RN01
    CONSTRAINT uq_usuario_gamertag UNIQUE (gamertag)             -- RN01
);

CREATE TABLE desenvolvedora (
    id_desenvolvedora INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nome              VARCHAR(100) NOT NULL,
    pais              VARCHAR(60),
    ano_fundacao      SMALLINT
);

CREATE TABLE publicadora (
    id_publicadora INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nome           VARCHAR(100) NOT NULL,
    pais           VARCHAR(60),
    ano_fundacao   SMALLINT
);

CREATE TABLE genero (
    id_genero INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nome      VARCHAR(50) NOT NULL,
    CONSTRAINT uq_genero_nome UNIQUE (nome)
);

CREATE TABLE plataforma (
    id_plataforma  INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nome           VARCHAR(50) NOT NULL,
    ano_lancamento SMALLINT,
    CONSTRAINT uq_plataforma_nome UNIQUE (nome)
);

CREATE TABLE jogo (
    id_jogo           INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    titulo            VARCHAR(150) NOT NULL,
    descricao         TEXT,
    data_lancamento   DATE         NOT NULL,
    id_desenvolvedora INTEGER      NOT NULL,                     -- RN03
    id_publicadora    INTEGER      NOT NULL,                     -- RN03
    CONSTRAINT fk_jogo_desenvolvedora FOREIGN KEY (id_desenvolvedora)
        REFERENCES desenvolvedora (id_desenvolvedora) ON DELETE RESTRICT,  -- RN19
    CONSTRAINT fk_jogo_publicadora FOREIGN KEY (id_publicadora)
        REFERENCES publicadora (id_publicadora) ON DELETE RESTRICT         -- RN19
);

CREATE TABLE jogo_genero (
    id_jogo   INTEGER NOT NULL,
    id_genero INTEGER NOT NULL,
    PRIMARY KEY (id_jogo, id_genero),
    CONSTRAINT fk_jogo_genero_jogo FOREIGN KEY (id_jogo)
        REFERENCES jogo (id_jogo) ON DELETE CASCADE,                     -- RN20
    CONSTRAINT fk_jogo_genero_genero FOREIGN KEY (id_genero)
        REFERENCES genero (id_genero) ON DELETE RESTRICT                 -- RN19
);

CREATE TABLE jogo_plataforma (
```

```
    id_jogo_plataforma         INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_jogo                    INTEGER NOT NULL,
    id_plataforma              INTEGER NOT NULL,
    data_lancamento_plataforma DATE,
    CONSTRAINT uq_jogo_plataforma UNIQUE (id_jogo, id_plataforma),
    CONSTRAINT fk_jogo_plataforma_jogo FOREIGN KEY (id_jogo)
        REFERENCES jogo (id_jogo) ON DELETE CASCADE,                      -- RN20
    CONSTRAINT fk_jogo_plataforma_plataforma FOREIGN KEY (id_plataforma)
        REFERENCES plataforma (id_plataforma) ON DELETE RESTRICT          -- RN19
);

CREATE TABLE pacote_conquistas (
    id_pacote          INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_jogo_plataforma INTEGER      NOT NULL,
    nome               VARCHAR(100) NOT NULL,
    tipo               VARCHAR(10)  NOT NULL,
    data_lancamento    DATE         NOT NULL,
    CONSTRAINT ck_pacote_tipo CHECK (tipo IN ('BASE', 'DLC', 'UPDATE')),   -- RN06
    CONSTRAINT fk_pacote_jogo_plataforma FOREIGN KEY (id_jogo_plataforma)
        REFERENCES jogo_plataforma (id_jogo_plataforma) ON DELETE CASCADE  -- RN20
);

-- RN06: no máximo um pacote BASE por jogo/plataforma
CREATE UNIQUE INDEX ux_pacote_base_por_jogo_plataforma
    ON pacote_conquistas (id_jogo_plataforma) WHERE tipo = 'BASE';

CREATE TABLE conquista (
    id_conquista INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_pacote    INTEGER      NOT NULL,
    nome         VARCHAR(100) NOT NULL,
    descricao    TEXT         NOT NULL,
    gamerscore   INTEGER      NOT NULL,
    CONSTRAINT ck_conquista_gamerscore CHECK (gamerscore > 0),             -- RN07
    CONSTRAINT fk_conquista_pacote FOREIGN KEY (id_pacote)
        REFERENCES pacote_conquistas (id_pacote) ON DELETE CASCADE         -- RN20
);

CREATE TABLE item_biblioteca (
    id_item            INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_usuario         INTEGER       NOT NULL,
    id_jogo_plataforma INTEGER       NOT NULL,                             -- RN10
    status             VARCHAR(15)   NOT NULL DEFAULT 'QUERO_JOGAR',
    horas_jogadas      NUMERIC(8,1)  NOT NULL DEFAULT 0,
    data_adicao        DATE          NOT NULL DEFAULT CURRENT_DATE,
    CONSTRAINT uq_item_usuario_jogo_plataforma
        UNIQUE (id_usuario, id_jogo_plataforma),                           -- RN11
    CONSTRAINT ck_item_status CHECK (status IN
        ('QUERO_JOGAR', 'JOGANDO', 'PAUSADO', 'ABANDONADO', 'FINALIZADO')),-- RN12
    CONSTRAINT ck_item_horas CHECK (horas_jogadas >= 0),                   -- RN13
    CONSTRAINT fk_item_usuario FOREIGN KEY (id_usuario)
        REFERENCES usuario (id_usuario) ON DELETE CASCADE,                 -- RN21
    CONSTRAINT fk_item_jogo_plataforma FOREIGN KEY (id_jogo_plataforma)
        REFERENCES jogo_plataforma (id_jogo_plataforma) ON DELETE CASCADE  -- RN20
);

CREATE TABLE desbloqueio (
    id_item      INTEGER   NOT NULL,
    id_conquista INTEGER   NOT NULL,
```

```
    data_hora    TIMESTAMP NOT NULL,                                    -- RN15
    PRIMARY KEY (id_item, id_conquista),                                -- RN15
    CONSTRAINT fk_desbloqueio_item FOREIGN KEY (id_item)
        REFERENCES item_biblioteca (id_item) ON DELETE CASCADE,         -- RN20, RN21
    CONSTRAINT fk_desbloqueio_conquista FOREIGN KEY (id_conquista)
        REFERENCES conquista (id_conquista) ON DELETE CASCADE           -- RN20
);

CREATE TABLE recomendacao (
    id_recomendacao INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_usuario      INTEGER      NOT NULL,
    data_geracao    TIMESTAMP    NOT NULL DEFAULT now(),
    modelo_ia       VARCHAR(80)  NOT NULL,
    CONSTRAINT fk_recomendacao_usuario FOREIGN KEY (id_usuario)
        REFERENCES usuario (id_usuario) ON DELETE CASCADE               -- RN21
);

CREATE TABLE recomendacao_jogo (
    id_recomendacao INTEGER  NOT NULL,
    id_jogo         INTEGER  NOT NULL,
    justificativa   TEXT     NOT NULL,                                  -- RN23
    posicao         SMALLINT NOT NULL,                                  -- RN23
    PRIMARY KEY (id_recomendacao, id_jogo),                             -- RN23
    CONSTRAINT uq_recomendacao_posicao UNIQUE (id_recomendacao, posicao),  -- RN23
    CONSTRAINT ck_recomendacao_posicao CHECK (posicao > 0),             -- RN23
    CONSTRAINT fk_recjogo_recomendacao FOREIGN KEY (id_recomendacao)
        REFERENCES recomendacao (id_recomendacao) ON DELETE CASCADE,    -- RN21
    CONSTRAINT fk_recjogo_jogo FOREIGN KEY (id_jogo)
        REFERENCES jogo (id_jogo) ON DELETE CASCADE                     -- RN20
);

-- ---------------------------------------------------------------------
-- 2. ÍNDICES DE APOIO ÀS CHAVES ESTRANGEIRAS
--    (o PostgreSQL não cria índice automático para FKs)
-- ---------------------------------------------------------------------
CREATE INDEX ix_jogo_desenvolvedora   ON jogo (id_desenvolvedora);
CREATE INDEX ix_jogo_publicadora      ON jogo (id_publicadora);
CREATE INDEX ix_jogo_genero_genero    ON jogo_genero (id_genero);
CREATE INDEX ix_jogo_plataforma_plat  ON jogo_plataforma (id_plataforma);
CREATE INDEX ix_pacote_jogo_plataforma ON pacote_conquistas (id_jogo_plataforma);
CREATE INDEX ix_conquista_pacote      ON conquista (id_pacote);
CREATE INDEX ix_item_jogo_plataforma  ON item_biblioteca (id_jogo_plataforma);
CREATE INDEX ix_desbloqueio_conquista ON desbloqueio (id_conquista);
CREATE INDEX ix_desbloqueio_data_hora ON desbloqueio (data_hora);
CREATE INDEX ix_recomendacao_usuario  ON recomendacao (id_usuario);
CREATE INDEX ix_recjogo_jogo          ON recomendacao_jogo (id_jogo);

-- ---------------------------------------------------------------------
-- 3. GATILHOS (TRIGGERS)
-- ---------------------------------------------------------------------

-- Limite de gamerscore por tipo de pacote (RN08 / RN09)
CREATE FUNCTION fn_limite_gamerscore(p_tipo VARCHAR) RETURNS INTEGER AS $$
    SELECT CASE WHEN p_tipo = 'BASE' THEN 1000 ELSE 250 END;
$$ LANGUAGE sql IMMUTABLE;

-- T1 (RN14): a conquista desbloqueada deve pertencer ao mesmo
-- jogo/plataforma do item da biblioteca.
```

```
CREATE FUNCTION fn_t1_desbloqueio_mesmo_jogo_plataforma() RETURNS trigger AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
          FROM item_biblioteca ib
          JOIN conquista c          ON c.id_conquista = NEW.id_conquista
          JOIN pacote_conquistas pc ON pc.id_pacote   = c.id_pacote
         WHERE ib.id_item = NEW.id_item
           AND pc.id_jogo_plataforma = ib.id_jogo_plataforma
    ) THEN
        RAISE EXCEPTION 'RN14: a conquista % não pertence ao jogo/plataforma do item de biblioteca
% (ou um deles não existe)',
            NEW.id_conquista, NEW.id_item
            USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_t1_desbloqueio_mesmo_jogo_plataforma
    BEFORE INSERT OR UPDATE OF id_item, id_conquista ON desbloqueio
    FOR EACH ROW EXECUTE FUNCTION fn_t1_desbloqueio_mesmo_jogo_plataforma();

-- T1 (complemento): impede trocar o vínculo jogo/plataforma de um item,
-- pacote ou conquista que já possui desbloqueios, o que quebraria a RN14.
CREATE FUNCTION fn_t1_protege_item() RETURNS trigger AS $$
BEGIN
    IF NEW.id_jogo_plataforma IS DISTINCT FROM OLD.id_jogo_plataforma
       AND EXISTS (SELECT 1 FROM desbloqueio WHERE id_item = OLD.id_item) THEN
        RAISE EXCEPTION 'RN14: o item de biblioteca % possui desbloqueios; não é possível alterar
o jogo/plataforma', OLD.id_item
            USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE FUNCTION fn_t1_protege_conquista() RETURNS trigger AS $$
BEGIN
    IF NEW.id_pacote IS DISTINCT FROM OLD.id_pacote
       AND EXISTS (SELECT 1 FROM desbloqueio WHERE id_conquista = OLD.id_conquista) THEN
        RAISE EXCEPTION 'RN14: a conquista % possui desbloqueios; não é possível mudá-la de
pacote', OLD.id_conquista
            USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE FUNCTION fn_t1_protege_pacote() RETURNS trigger AS $$
BEGIN
    IF NEW.id_jogo_plataforma IS DISTINCT FROM OLD.id_jogo_plataforma
       AND EXISTS (SELECT 1 FROM conquista c
                     JOIN desbloqueio d ON d.id_conquista = c.id_conquista
                    WHERE c.id_pacote = OLD.id_pacote) THEN
        RAISE EXCEPTION 'RN14: o pacote % possui conquistas desbloqueadas; não é possível alterar
o jogo/plataforma', OLD.id_pacote
            USING ERRCODE = 'check_violation';
```

```
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_t1_protege_item
    BEFORE UPDATE OF id_jogo_plataforma ON item_biblioteca
    FOR EACH ROW EXECUTE FUNCTION fn_t1_protege_item();
CREATE TRIGGER trg_t1_protege_conquista
    BEFORE UPDATE OF id_pacote ON conquista
    FOR EACH ROW EXECUTE FUNCTION fn_t1_protege_conquista();
CREATE TRIGGER trg_t1_protege_pacote
    BEFORE UPDATE OF id_jogo_plataforma ON pacote_conquistas
    FOR EACH ROW EXECUTE FUNCTION fn_t1_protege_pacote();

-- T2 (RN15): a data/hora do desbloqueio não pode ser futura.
CREATE FUNCTION fn_t2_data_hora_nao_futura() RETURNS trigger AS $$
BEGIN
    IF NEW.data_hora > LOCALTIMESTAMP THEN
        RAISE EXCEPTION 'RN15: a data/hora do desbloqueio (%) não pode ser futura', NEW.data_hora
            USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_t2_data_hora_nao_futura
    BEFORE INSERT OR UPDATE OF data_hora ON desbloqueio
    FOR EACH ROW EXECUTE FUNCTION fn_t2_data_hora_nao_futura();

-- T3 (RN08 / RN09): a soma do gamerscore de um pacote não pode exceder
-- 1000 G (BASE) ou 250 G (DLC/UPDATE). A igualdade a 1000 G no pacote
-- BASE é verificada pela visão vw_auditoria_integridade.
CREATE FUNCTION fn_t3_gamerscore_conquista() RETURNS trigger AS $$
DECLARE
    v_tipo   VARCHAR(10);
    v_outras INTEGER;
BEGIN
    -- trava o pacote para serializar inserções concorrentes na mesma soma
    SELECT tipo INTO v_tipo FROM pacote_conquistas
     WHERE id_pacote = NEW.id_pacote FOR UPDATE;
    IF NOT FOUND THEN
        RETURN NEW;                     -- a FK reportará o erro
    END IF;

    SELECT COALESCE(SUM(gamerscore), 0) INTO v_outras
      FROM conquista
     WHERE id_pacote = NEW.id_pacote AND id_conquista <> NEW.id_conquista;

    IF v_outras + NEW.gamerscore > fn_limite_gamerscore(v_tipo) THEN
        RAISE EXCEPTION 'RN08/RN09: o pacote % (%) excederia % G (soma atual: % G + nova
conquista: % G)',
            NEW.id_pacote, v_tipo, fn_limite_gamerscore(v_tipo), v_outras, NEW.gamerscore
            USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_t3_gamerscore_conquista
    BEFORE INSERT OR UPDATE OF gamerscore, id_pacote ON conquista
    FOR EACH ROW EXECUTE FUNCTION fn_t3_gamerscore_conquista();

-- T3 (complemento): ao mudar o tipo de um pacote, o novo limite também
-- precisa ser respeitado pela soma já existente.
CREATE FUNCTION fn_t3_tipo_pacote() RETURNS trigger AS $$
DECLARE
    v_soma INTEGER;
BEGIN
    SELECT COALESCE(SUM(gamerscore), 0) INTO v_soma
      FROM conquista WHERE id_pacote = NEW.id_pacote;
    IF v_soma > fn_limite_gamerscore(NEW.tipo) THEN
        RAISE EXCEPTION 'RN08/RN09: o pacote % possui % G, acima do limite de % G para o tipo %',
            NEW.id_pacote, v_soma, fn_limite_gamerscore(NEW.tipo), NEW.tipo
            USING ERRCODE = 'check_violation';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_t3_tipo_pacote
    BEFORE UPDATE OF tipo ON pacote_conquistas
    FOR EACH ROW EXECUTE FUNCTION fn_t3_tipo_pacote();

-- ---------------------------------------------------------------------
-- 4. VISÃO DE AUDITORIA
--    Regras de completude ("exatamente 1000 G", "ao menos um") que não
--    podem ser impostas a cada inserção isolada. Linhas retornadas
--    indicam violações; resultado vazio = banco consistente.
-- ---------------------------------------------------------------------
CREATE VIEW vw_auditoria_integridade AS
SELECT 'RN08'::VARCHAR(10) AS regra,
       'Pacote BASE com gamerscore total diferente de 1000'::TEXT AS problema,
       ('pacote ' || pc.id_pacote || ' (' || pc.nome || '): '
            || COALESCE(SUM(c.gamerscore), 0) || ' G')::TEXT AS referencia
  FROM pacote_conquistas pc
  LEFT JOIN conquista c ON c.id_pacote = pc.id_pacote
 WHERE pc.tipo = 'BASE'
 GROUP BY pc.id_pacote, pc.nome
HAVING COALESCE(SUM(c.gamerscore), 0) <> 1000
UNION ALL
SELECT 'RN06', 'Jogo/plataforma sem pacote BASE',
       'jogo_plataforma ' || jp.id_jogo_plataforma || ' (' || j.titulo || ' / ' || p.nome || ')'
  FROM jogo_plataforma jp
  JOIN jogo j       ON j.id_jogo = jp.id_jogo
  JOIN plataforma p ON p.id_plataforma = jp.id_plataforma
 WHERE NOT EXISTS (SELECT 1 FROM pacote_conquistas pc
                   WHERE pc.id_jogo_plataforma = jp.id_jogo_plataforma AND pc.tipo = 'BASE')
UNION ALL
SELECT 'RN04', 'Jogo sem gênero', 'jogo ' || j.id_jogo || ' (' || j.titulo || ')'
  FROM jogo j
 WHERE NOT EXISTS (SELECT 1 FROM jogo_genero jg WHERE jg.id_jogo = j.id_jogo)
UNION ALL
SELECT 'RN04', 'Jogo sem plataforma', 'jogo ' || j.id_jogo || ' (' || j.titulo || ')'
  FROM jogo j
 WHERE NOT EXISTS (SELECT 1 FROM jogo_plataforma jp WHERE jp.id_jogo = j.id_jogo)
```

```
UNION ALL
SELECT 'RN22', 'Recomendação sem nenhum jogo', 'recomendacao ' || r.id_recomendacao
  FROM recomendacao r
 WHERE NOT EXISTS (SELECT 1 FROM recomendacao_jogo rj WHERE rj.id_recomendacao =
r.id_recomendacao);
```

5.3 Suporte a embeddings (02_pgvector.sql)

A recomendação por IA (seção 7.3) usa o pgvector para buscar jogos semelhantes. As duas colunas acrescentadas à tabela `jogo` são detalhes de implementação e não fazem parte do modelo lógico: `embedding` guarda o vetor de 768 dimensões do jogo, e `url_capa` guarda o endereço da imagem exibida na interface (pode ficar nulo).

```
-- ======================================================================
-- Suporte a embeddings (pgvector) - item 7f (IA generativa)
-- Executar APÓS 01_schema.sql.
-- A dimensão do vetor (768) corresponde ao modelo gemini-embedding-001 com
-- output_dimensionality=768. Se trocar de modelo, recrie a coluna.
-- ======================================================================
CREATE EXTENSION IF NOT EXISTS vector;

ALTER TABLE jogo ADD COLUMN embedding vector(768);

-- Detalhe de implementação (não faz parte do modelo lógico): endereço da imagem
-- de capa, usado apenas para exibição na interface. Pode ficar nulo.
ALTER TABLE jogo ADD COLUMN url_capa TEXT;

-- Com algumas dezenas de jogos a busca sequencial basta. Para volumes
-- maiores, um índice aproximado (HNSW, distância de cosseno) seria:
-- CREATE INDEX ix_jogo_embedding ON jogo USING hnsw (embedding vector_cosine_ops);
```

6. Consultas

As três consultas a seguir agregam dados de três ou mais tabelas. Os resultados abaixo foram obtidos sobre a carga inicial (`03_carga.sql`) e os gráficos são gerados pela própria aplicação (página Consultas), que também permite baixá-los em PNG.

6.1 Ranking de gamerscore por usuário

Descrição. Soma o gamerscore das conquistas desbloqueadas por cada usuário e ordena do maior para o menor. Responde: quem são os jogadores mais conquistadores da plataforma?

Tabelas envolvidas. `usuario`, `item_biblioteca`, `desbloqueio`, `conquista`

Comando SQL.

```
SELECT u.gamertag,
       COUNT(*)          AS conquistas_desbloqueadas,
       SUM(c.gamerscore) AS gamerscore_total
  FROM usuario u
  JOIN item_biblioteca ib ON ib.id_usuario = u.id_usuario
  JOIN desbloqueio d      ON d.id_item = ib.id_item
  JOIN conquista c        ON c.id_conquista = d.id_conquista
 GROUP BY u.id_usuario, u.gamertag
 ORDER BY gamerscore_total DESC, u.gamertag
```

Resultado.

gamertag	conquistas_desbloqueadas	gamerscore_total
CH_Lima88	55	4870
JuPradoGG	54	4750
AnaBeaXbox	46	3900
LariPlays	39	3065
RafaSouza_BR	31	2550

Gráfico.

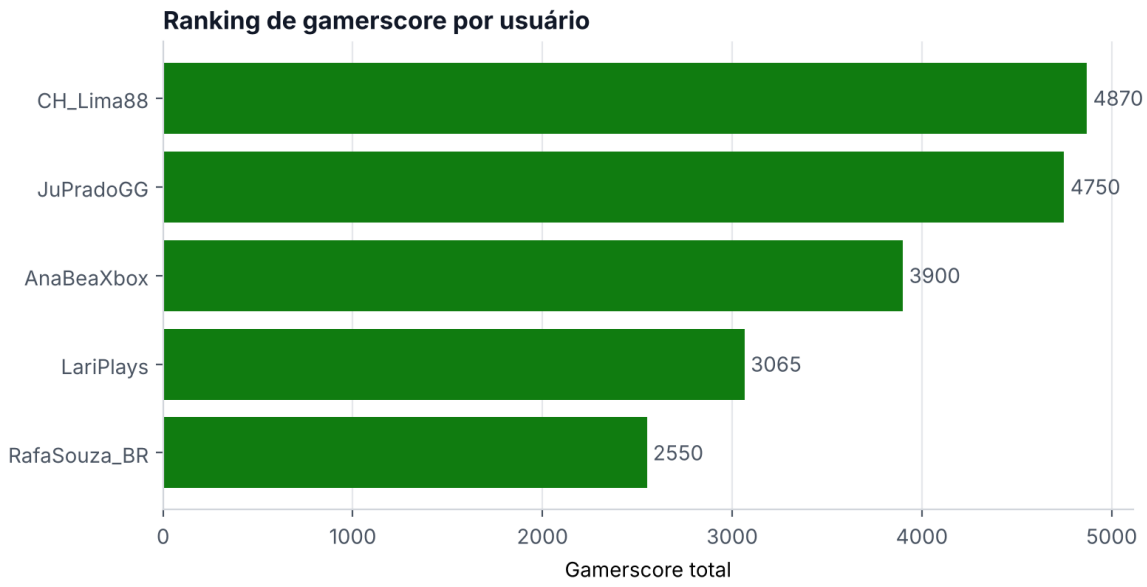


Figura 3: Ranking de gamerscore por usuário

Análise. CH_Lima88 lidera com 4870 G em 55 conquistas, seguido de perto por JuPradoGG (4750 G). RafaSouza_BR, com 2550 G, tem pouco mais da metade do primeiro colocado.

6.2 Conquistas desbloqueadas por trimestre

Descrição. Agrupa os desbloqueios por trimestre do ano, contando as conquistas, somando o gamerscore e o número de usuários ativos. Responde: como a atividade da plataforma evolui ao longo do tempo?

Tabelas envolvidas. `desbloqueio`, `item_biblioteca`, `conquista`

Comando SQL.

```
SELECT EXTRACT(YEAR FROM d.data_hora)::int || '-T' || EXTRACT(QUARTER FROM d.data_hora)::int AS
trimestre,
       COUNT(*)                        AS conquistas,
       SUM(c.gamerscore)               AS gamerscore,
       COUNT(DISTINCT ib.id_usuario)   AS usuarios_ativos
  FROM desbloqueio d
  JOIN item_biblioteca ib ON ib.id_item = d.id_item
  JOIN conquista c        ON c.id_conquista = d.id_conquista
 GROUP BY 1
 ORDER BY 1
```

Resultado.

trimestre	conquistas	gamerscore	usuarios_ativos
2024-T1	33	2495	4

trimestre	conquistas	gamerscore	usuarios_ativos
2024-T2	37	3780	4
2024-T3	19	1825	4
2024-T4	10	900	2
2025-T1	33	3075	5
2025-T2	17	1500	4
2025-T3	15	885	4
2025-T4	16	1400	5
2026-T1	24	1700	4
2026-T2	12	1060	2
2026-T3	7	315	1
2026-T4	2	200	1

Gráfico.

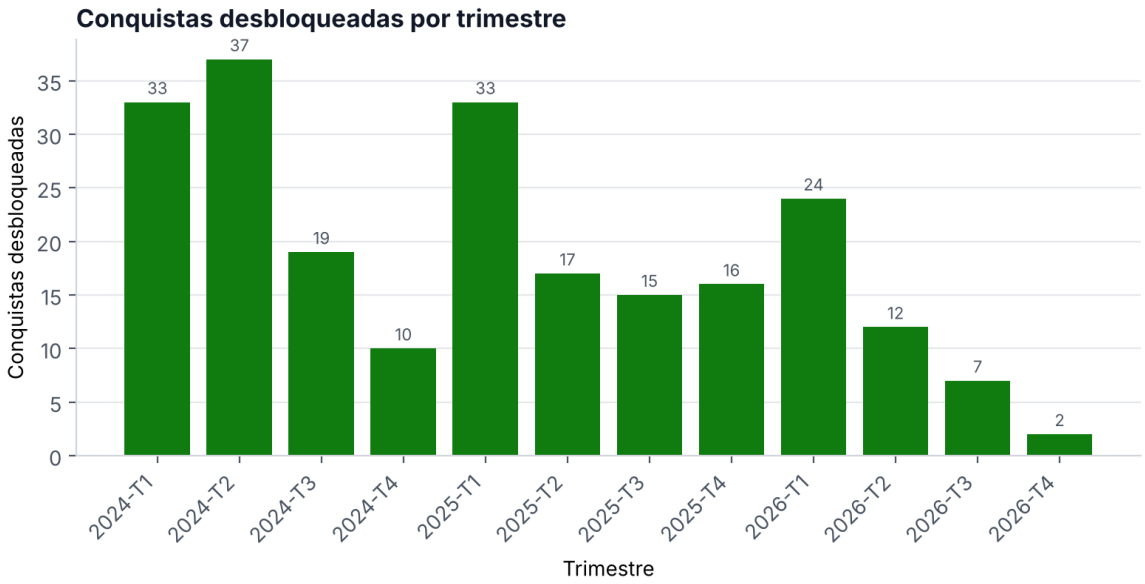


Figura 4: Conquistas desbloqueadas por trimestre

Análise. A atividade se concentra nos dois primeiros trimestres de 2024 (33 e 37 conquistas) e em 2025-T1 (33), e diminui depois: em 2026-T3 e 2026-T4 restam apenas um usuário ativo e poucas conquistas, o que reflete a data limite dos dados de teste.

6.3 Jogos concluídos (100% do pacote base) por ano

Descrição. Considera concluído o item da biblioteca cujo usuário desbloqueou todas as conquistas do pacote BASE; o ano é o da última conquista desbloqueada. Responde: quantos jogos foram finalizados a cada ano?

Tabelas envolvidas. `item_biblioteca`, `pacote_conquistas`, `conquista`, `desbloqueio`

Comando SQL.

```
WITH base AS (
    SELECT ib.id_item,
           COUNT(c.id_conquista) AS total,
           COUNT(d.id_conquista) AS desbloqueadas,
           MAX(d.data_hora)      AS ultima
      FROM item_biblioteca ib
      JOIN pacote_conquistas pc ON pc.id_jogo_plataforma = ib.id_jogo_plataforma
                               AND pc.tipo = 'BASE'
      JOIN conquista c          ON c.id_pacote = pc.id_pacote
      LEFT JOIN desbloqueio d   ON d.id_item = ib.id_item
                               AND d.id_conquista = c.id_conquista
     GROUP BY ib.id_item
)
SELECT EXTRACT(YEAR FROM ultima)::int AS ano,
       COUNT(*)                       AS jogos_concluidos
  FROM base
 WHERE total > 0 AND desbloqueadas = total
 GROUP BY 1
 ORDER BY 1
```

Resultado.

ano	jogos_concluidos
2024	7
2025	6
2026	2

Gráfico.

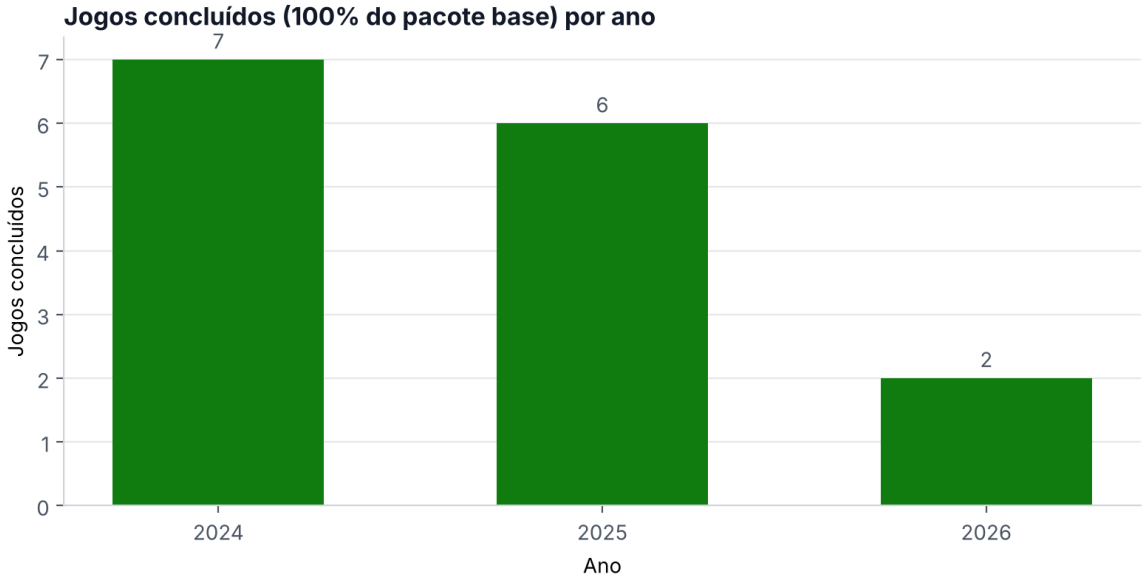


Figura 5: Jogos concluídos (100% do pacote base) por ano

Análise. Foram concluídos 7 jogos em 2024, 6 em 2025 e 2 em 2026, ou seja, 15 conclusões no total, com queda ao longo do tempo acompanhando o padrão da consulta anterior.

7. Aplicação

A aplicação é uma interface web local, escrita em Python com Streamlit, que se conecta diretamente ao PostgreSQL e cobre todas as funcionalidades exigidas. O código-fonte está no repositório do projeto:

https://github.com/moku-sd/banco-de-dados-i/tree/main/trabalho-final

7.1 Tecnologias e execução

Componente	Uso
PostgreSQL 18 com pgvector	Banco de dados, em contêiner Docker (pgvector/pgvector:pg18), junto com o pgAdmin
Python e psycopg2	Acesso ao banco com consultas parametrizadas, pool de conexões e transações
Streamlit	Interface web: menu lateral, formulários, tabelas e gráficos
pandas e matplotlib	Resultado das consultas em tabela e gráficos (exportáveis em PNG)
Gemini (google-genai)	Embeddings (gemini-embedding-001, 768 dimensões) e geração das justificativas

Para executar, na pasta do projeto: `docker compose up -d`, `pip install -r requirements.txt` e `streamlit run src/app.py`. As credenciais do banco e a chave do Gemini ficam no arquivo `.env`, fora do código-fonte e do repositório (RNF05).

7.2 Funcionalidades

Funcionalidade	Onde na aplicação	Item
Conexão com o PostgreSQL	src/db.py (parâmetros do .env, pool de conexões)	7a
Inserir, consultar, alterar e excluir em todas as 14 tabelas	Página Cadastros (formulários gerados a partir da descrição das tabelas, com listas suspensas nas chaves estrangeiras) e página Biblioteca	7b
Execução das 3 consultas, com tabela e gráfico	Página Consultas (mostra o SQL, a tabela e o gráfico; exporta PNG e CSV)	7c
Menu principal	Menu lateral com as cinco páginas	7d
Criar tabelas, carregar dados e eliminar tudo	Página Banco de dados (parte de um banco vazio; botões Criar, Carregar, Eliminar e Recriar tudo)	7e
Recomendação com IA generativa	Página Recomendação IA	7f

Nas operações de escrita, as dependências são respeitadas pela ordem dos cadastros (uma lista suspensa só oferece registros já existentes) e pelas regras do

próprio banco. Quando uma restrição ou um gatilho recusa a operação, a aplicação traduz o erro do PostgreSQL em uma mensagem que cita a regra de negócio, por exemplo "RN15: a data/hora do desbloqueio não pode ser futura" ou "RN19: não é possível excluir, o registro ainda é usado na tabela jogo". A página Biblioteca mostra, para cada jogo do usuário, a capa, o status, as horas, a barra de progresso (conquistas desbloqueadas sobre o total) e o gamerscore obtido.

7.3 Recomendação com IA generativa

A recomendação segue um esquema de duas etapas, em que o banco de dados escolhe os candidatos e o modelo de linguagem apenas decide entre eles e explica a escolha:

1. Embeddings. Para cada jogo é montado um texto (título, gêneros, desenvolvedora e descrição), convertido em um vetor de 768 dimensões pelo modelo `gemini-embedding-001` e guardado em `jogo.embedding`.

2. Perfil e recuperação (pgvector). O perfil do usuário é a média dos vetores dos jogos que ele finalizou ou está jogando. Uma consulta ordena os jogos pela distância de cosseno (operador <=>) em relação ao perfil e devolve os mais próximos que ainda não estão na biblioteca dele (os candidatos).

3. Geração (Gemini). O modelo recebe o histórico do usuário (jogos, plataformas, status, horas e gêneros) e somente os candidatos, escolhe os melhores em ordem e escreve uma justificativa em português para cada um.

4. Validação e gravação. A resposta é recebida em JSON e validada: só são aceitos jogos que estão entre os candidatos, sem repetição e com justificativa. O resultado é gravado em uma única transação em `recomendacao` (com o modelo que respondeu) e em `recomendacao_jogo` (posição e justificativa), o que garante a regra RN22 de que toda recomendação lista ao menos um jogo.

Como o modelo não pode sugerir jogos fora da lista do banco, a recomendação é sempre explicável: a página mostra os candidatos com a similaridade calculada e o texto enviado ao modelo.